

≡ Hide menu

GPU Programming
Specialization

- ✓

Reading: Course Overview
10 min
- 📖

Reading: Course Outline
10 min
- ▶

Video: GPU Programming
Specialization
3 min
- 💬

Discussion Prompt: Large
Scale Data and Challenges
Discussion
15 min

Course Technical and
Programming
Expectations

Coursera Lab and
Assignment Usage

Course Overview

Description

The purpose of this course is to instill in you core programming concepts required from Nvidia CUDA programming. These topics are computational model and memory for CUDA hardware/software architectures. Understanding of the model of threads, blocks, and grids, will be used in the definition how the GPU will configure parallelization in hardware. You will need to know the pros and cons of the various types of device memory and how they are used in different architectures and even individual GPU card models. This will aid in ensuring that your software runs and performs as you desire. The complexity and topics of the laboratory activities and assignments, will challenge you to understand canonical data processing patterns and how they are developed. Finally, you will be presented with an initial venture into memory analysis, which will be important for software performance optimization.

At the very end of the course, you should be able to take a given CUDA development/production environment, which includes specific Nvidia GPU devices and CUDA versions, and determine what the computational and memory capabilities and constraints are. This will be used in software design and implementation. If you have not encountered C++ memory allocation and management, you will gain an understanding of this which will be very helpful when you are developing general C++ software. While the level of programming may be lower than what you may be accustomed to, it will be introduced in a gradual and incremental fashion, so that you have an opportunity to fill in any holes in your understanding of the C/C++ programming language.

Target Students

You should have a desire to begin or continue the path to mastery of parallel programming techniques and GPU programming. You can come from a systems administration, software development, data science, or ML/AI background. You will need a base understanding C/C++, which should grow throughout the course. There is no requirement to be a professional C++ programmer. Through this course and the others in this specialization, you should become comfortable with developing C/C++ programs that use GPUs.

Requirements

- Basic Programming knowledge of C/C++
- Understanding of operating system command line terminals, e.g. Window's CMD and macOS' Terminal applications
- Experience using text editors and/or integrated development environments
- General understanding of CPU/GPU hardware systems, including processors, registers, memory, cache, etc.

Optional Technical Knowledge:

- Linux
- Make
- Bash, Zsh, and other Linux shells
- Debuggers and REPLs

Expectations

The difficulty of the programming will grow throughout this course. You will be given the scaffolding for all programming assignments and labs. You will be given instructions on how to add/modify code to complete assignments, though you should feel free to modify and augment the logic. When modifying the existing code you will need to ensure that you do not change the output to files and standard I/O, as this is used to grade the assignment. Also, you may have to use different programming patterns that you are used to, e.g. char pointers vs strings. Your responsibility will be to learn how to write your code to meet the interfaces/patterns of the code or modify the harness code to use the syntax that you are used to.

You will need to complete quizzes to verify your knowledge of important concepts. Quiz questions will be based on content in the various video lectures, so pay attention to the various information presented there. You will also have discussions for all weeks/modules, the can be used to share your experience with others and also feel free to ask technical questions of those who are also taking the course. If there are significant technical issues with any material, utilize the **Course Technical Issues** discussion forum, as it will be checked periodically to allow for fixing of bugs, etc. Just like everything in programming, there may be helpful information on internet resources such as Google and StackOverflow, but be careful in using them completely, as they may not apply to your specific issue and this may cause you to break the Coursera Honor Code.

Labs vs Assignment

This course contains many ungraded labs and assignments. Lab activities are opportunities to explore programs and apply the knowledge that you are learning in a particular module; they are not graded. Programming assignments have been developed to be graded by following the instructions. The student will need to execute and submit their work via custom built buttons at the bottom of the web page. This is different then the Submit button that appears close to the top right of the web page, which will not appropriately submit your work. When you submit via the appropriate button at the bottom of the page, you will be informed via the terminal if the assignment results were submitted for automatic grading. You will need to look at the assignment's submission tab for the results of the latest submission. Use the feedback to improve your submission. You will need to complete all assignments successfully, along with quizzes, to pass the course.

Certificate Disclaimer

Upon completion of this course you will receive a certificate. This certificate does not affirm that this learner was enrolled as a student at Johns Hopkins University. It does not confer a JHU grade, course credit or degree; establish a relationship between this learner and JHU; enroll or register this learner at JHU or in any course offered by JHU; or entitle this learner to access or use resources beyond the online courses provided by Coursera.

Go to next item ✓ Completed

👍 Like 💬 Dislike 📝 Report an issue

